HDR Imagery Dynamic Tone Mapping OpenFX Plugin for DaVinci Resolve Design Specification

Group 4

May 2026

Contents

This Design Specification provides a concise breakdown of the major parts, modules, tools, and components planned for the *HDR Imagery Dynamic Tone Mapping: OpenFX Plugin for DaVinci Resolve* project. It expands on the Software Design Document (SDD) by describing how each planned part of the plugin is organized, what each component is responsible for, and how the user interacts with the system inside the DaVinci Resolve workflow.

1 System Parts and Modules

1.1 OpenFX Host Integration Module (OHIM)

Purpose

The OpenFX Host Integration Module manages communication between DaVinci Resolve and the HDR Dynamic Tone Mapping plugin. This module allows the plugin to be loaded as an OpenFX effect, receive image frame data from the host application, expose user-adjustable parameters, and return the processed frame back to DaVinci Resolve for preview and export.

Components / Tools Used

- OpenFX plugin framework
- DaVinci Resolve host application
- Plugin parameter registration system
- Frame request and render callback structure

Inputs and Outputs

- **Inputs:** HDR video frames, timeline context, user-selected plugin parameters
- **Outputs:** processed video frames, parameter updates, preview-ready image data

1.2 Frame Input Handler (FIH)

Purpose

The Frame Input Handler receives source frames from DaVinci Resolve and prepares them for analysis and processing. It is responsible for preserving the original image data, identifying the expected color format, and passing frame information to the luminance analysis and tone mapping modules.

Components / Tools Used

- Frame buffer input handler
- Pixel format validation
- Color space identification logic
- HDR metadata access, when available

Inputs and Outputs

- **Inputs:** raw HDR frame data from DaVinci Resolve
- **Outputs:** validated frame data, detected color space information, prepared image buffer

1.3 HDR Metadata and Color Space Module (HMCSM)

Purpose

The HDR Metadata and Color Space Module identifies the technical properties of the incoming footage. This includes the input color space, expected output target, and available HDR metadata. If metadata is unavailable, the module provides fallback behavior by allowing the tone mapping system to estimate luminance characteristics directly from the frame.

Components / Tools Used

- Color space profile definitions
- Input target options such as Rec.709, Rec.2020, and P3-D65
- Output target options such as SDR, HDR10, and custom output
- Metadata fallback logic

Inputs and Outputs

- **Inputs:** frame data, project color settings, HDR metadata, user-selected output target
- **Outputs:** normalized color information, output target settings, metadata warnings when needed

1.4 Luminance Analysis Module (LAM)

Purpose

The Luminance Analysis Module evaluates the brightness characteristics of a frame or scene. It calculates values such as peak brightness, average luminance, shadow density, highlight intensity, and midtone distribution. These values are used by the tone mapping engine to produce a balanced output.

Components / Tools Used

- Peak brightness detector
- Average luminance calculator
- Shadow and highlight range analyzer
- Scene-adaptive brightness evaluation

Inputs and Outputs

- **Inputs:** validated HDR frame data from the Frame Input Handler
- **Outputs:** luminance statistics, highlight range data, shadow range data, scene exposure profile

1.5 Dynamic Tone Mapping Engine (DTME)

Purpose

The Dynamic Tone Mapping Engine is the core processing component of the plugin. It compresses high dynamic range brightness values into a target display range while attempting to preserve highlight detail, shadow detail, contrast, and the creative intent of the image. The engine combines luminance analysis results with the user's selected parameters or preset values.

Components / Tools Used

- Tone mapping curve generator
- Highlight roll-off control
- Shadow recovery control
- Exposure and contrast adjustment logic
- Saturation protection logic
- Output range mapper

Inputs and Outputs

- **Inputs:** luminance statistics, selected preset, manual user controls, output target
- **Outputs:** tone-mapped image frame prepared for preview or export

1.6 Preset Manager Module (PMM)

Purpose

The Preset Manager Module stores and applies predefined tone mapping configurations. Presets allow users to quickly apply common image-processing styles without manually adjusting each control. The Custom preset allows users to override default values and fine-tune the output manually.

Components / Tools Used

- Natural preset
- Cinematic preset
- Broadcast Safe preset
- High Contrast preset
- Custom preset
- Preset-to-parameter mapping table

Inputs and Outputs

- **Inputs:** user preset selection
- **Outputs:** exposure, contrast, highlight roll-off, shadow recovery, and saturation values

1.7 Preview and Comparison Module (PCM)

Purpose

The Preview and Comparison Module controls how users view the processed result inside DaVinci Resolve. It supports full preview and comparison modes so that editors can evaluate the original HDR input against the tone-mapped output before applying final changes.

Components / Tools Used

- Full preview mode
- Split-view comparison mode
- Side-by-side comparison mode
- Original input preview
- Processed output preview

Inputs and Outputs

- **Inputs:** original frame, processed frame, selected preview mode
- **Outputs:** preview frame displayed in DaVinci Resolve

1.8 Error Handling and Validation Module (EHVM)

Purpose

The Error Handling and Validation Module manages invalid settings, unsupported inputs, missing HDR metadata, and processing failures. The plugin is designed to fail safely by warning the user and returning a usable image rather than producing corrupted output.

Components / Tools Used

- Unsupported format warnings
- Missing metadata fallback behavior
- Default SDR output target selection
- Invalid preset recovery
- Original-frame fallback behavior

Inputs and Outputs

- **Inputs:** system errors, invalid user settings, unsupported format conditions
- **Outputs:** warning messages, default settings, fallback preview frame

2 Directory Structure

The project repository is organized to support documentation, planning, testing, and mock development artifacts. Since this project focuses on software engineering documentation and workflow, the directory structure separates design documents, requirements, user documentation, testing artifacts, and mock plugin resources.

2.1 Root Directory

The root directory contains the main project README, Docker configuration, and top-level folders for documentation and mock plugin resources.

- `README.md` – Provides the project overview, Jira link, objectives, tools used, and repository guide.
- `docker-compose.yml` – Defines the planned development, documentation, and testing environments.
- `docs/` – Stores all project documentation.
- `mock_plugin/` – Represents the planned source location for future OpenFX plugin implementation.
- `jira/` – Stores sprint planning summaries and task breakdowns.

2.2 Documentation Directory

The `docs/` directory contains the major documents required for the project.

- `Software_Design_Document/` – Contains SDD versions for each snapshot.
- `Software_Requirement_Specification/` – Contains SRS versions for each snapshot.
- `Design_Specification/` – Contains this design specification document.
- `User_Manual/` – Contains the user manual and usage explanation.
- `Snapshot_Objectives/` – Contains objective and reflection documents for each snapshot.
- `TestRail_Documents/` – Contains TestRail test cases and test run summaries.
- `Workflow_Diagram/` – Contains the high-level plugin workflow diagram.

2.3 Mock Plugin Directory

The `mock_plugin/` directory represents where implementation files would be stored in a future development phase. It may include placeholder files, planned module descriptions, and notes about how the OpenFX plugin would eventually be structured.

3 Tools and Technologies

This section provides a concise breakdown of the tools and technologies used or planned for the project.

3.1 DaVinci Resolve

DaVinci Resolve is the target host application for the plugin. The plugin is designed to fit into the existing DaVinci Resolve editing and color correction workflow as an OpenFX effect.

3.2 OpenFX Plugin Framework

OpenFX is the plugin framework used to connect the HDR tone mapping tool to DaVinci Resolve. It defines how the host application loads the plugin, sends frame data, exposes parameters, and receives processed output.

3.3 C++

C++ is the planned implementation language for performance-critical plugin development. Image processing operations such as luminance analysis and tone mapping would require efficient frame-level computation.

3.4 Docker

Docker is used to define a consistent mock development environment. The Docker Compose file describes planned services such as a plugin build environment, LaTeX documentation builder, and mock testing environment.

3.5 LaTeX

LaTeX is used to write and format formal project documents such as the SDD, SRS, Design Specification, Snapshot Objectives, and User Manual.

3.6 GitHub

GitHub is used for version control, document storage, branch management, pull requests, and project collaboration.

3.7 Jira

Jira is used for sprint planning, task assignment, bug tracking, and progress documentation across each project snapshot.

3.8 TestRail

TestRail is used to organize test cases and test run reports for snapshot checkpoints. TestRail cases are used to verify planned behavior such as tone mapping controls, preset selection, preview modes, and error handling.

4 User Interface Summary

The plugin user interface is designed to appear as a parameter panel inside DaVinci Resolve after the user applies the OpenFX effect to a clip. The interface is organized so that basic users can quickly select a preset, while advanced users can fine-tune individual tone mapping controls.

4.1 Main Plugin Panel

The main plugin panel contains the primary controls for enabling and configuring tone mapping.

- **Enable Dynamic Tone Mapping:** Turns the tone mapping process on or off.
- **Input Color Space:** Allows the user to select or confirm the input color space.
- **Output Target:** Allows the user to choose SDR, HDR10, or a custom display target.
- **Preset:** Allows the user to select Natural, Cinematic, Broadcast Safe, High Contrast, or Custom.

4.2 Tone Mapping Controls

The tone mapping controls allow users to manually adjust the image after selecting a preset or enabling dynamic mapping.

- **Exposure:** Adjusts overall image brightness.
- **Highlight Roll-Off:** Controls how bright highlights are compressed.
- **Shadow Recovery:** Restores detail in darker areas of the image.
- **Contrast:** Adjusts separation between light and dark regions.
- **Saturation Protection:** Reduces oversaturation caused by brightness compression.
- **Peak Brightness Target:** Defines the intended brightness range for the output display.

4.3 Preview Controls

Preview controls allow users to compare the original HDR image with the processed tone-mapped output.

- **Full Preview:** Displays only the processed output.
- **Split View:** Displays the original and processed result in one frame.
- **Side-by-Side:** Displays the original and processed images next to each other.
- **Reset Settings:** Restores plugin controls to their default state.

5 Processing Workflow

The processing workflow describes how frame data moves through the plugin.

1. The user imports HDR footage into DaVinci Resolve.
2. The user applies the HDR Dynamic Tone Mapping OpenFX plugin to a clip.
3. DaVinci Resolve sends the requested frame to the plugin through the OpenFX interface.
4. The Frame Input Handler validates the frame and identifies the expected format.
5. The HDR Metadata and Color Space Module checks color space and metadata information.
6. The Luminance Analysis Module calculates brightness and contrast statistics.
7. The Dynamic Tone Mapping Engine applies the selected tone mapping curve and user controls.
8. The Preview and Comparison Module prepares the processed frame for display.
9. DaVinci Resolve displays the processed output to the user.
10. The user adjusts settings or exports the final corrected video.

6 Preset Behavior Summary

Preset	Behavior
Natural	Produces a balanced tone mapping result intended to preserve a realistic image with moderate highlight compression and shadow recovery.
Cinematic	Increases contrast and applies stronger highlight roll-off for a more stylized video look.
Broadcast Safe	Prioritizes output values that are safer for SDR delivery and avoids excessive brightness or saturation.
High Contrast	Produces stronger separation between shadows and highlights while maintaining controlled peak brightness.
Custom	Allows the user to manually control all available tone mapping settings.

7 Error Handling Summary

Condition	System Response
Unsupported input color space	Display a warning and apply a default color profile.
Missing HDR metadata	Estimate brightness values directly from the frame data.
Invalid preset configuration	Revert to the Natural preset and notify the user.
Frame processing failure	Return the original frame to avoid corrupted output.
No output target selected	Default to SDR output targeting.
Extreme highlight clipping detected	Apply stronger highlight roll-off and display a warning if needed.

8 Testing Considerations

The design will be verified through TestRail test cases that validate the major parts of the plugin workflow. Testing will focus on whether the planned system behavior matches the requirements described in the Software Requirements Specification.

- Verify that the plugin can be applied to HDR footage inside DaVinci Resolve.
- Verify that users can enable and disable dynamic tone mapping.
- Verify that preset selection updates tone mapping parameters.
- Verify that manual controls adjust the processed image as expected.
- Verify that preview comparison modes display the correct output.
- Verify that unsupported inputs trigger safe warning behavior.
- Verify that missing HDR metadata does not prevent frame processing.

9 Future Design Improvements

Future versions of the plugin may include GPU acceleration, real-time histogram displays, Dolby Vision metadata support, advanced color space conversion, batch processing support, machine learning-based tone mapping, and shared preset libraries for collaborative post-production workflows.